TECHNISCHE
UNIVERSITÄT
MÜNCHEN

Software Engineering in Industrial Practice

(SEIP)

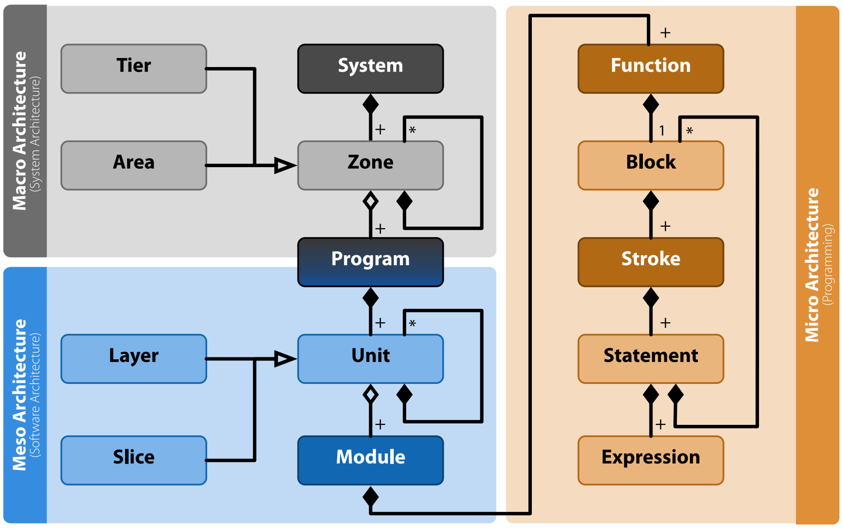
Dr. Ralf S. Engelschall

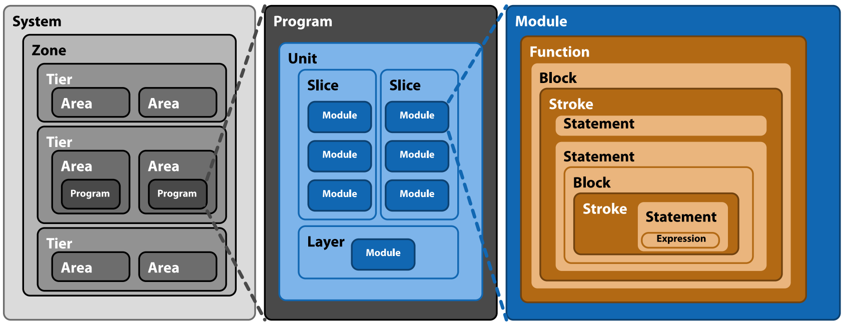
ARCHITECTURE FUNDAMENTALS

Component Hierarchy

TECHNISCHE UNIVERSITÄT MÜNCHEN

Intellectual Content: Version 1.2.3 (2022-11-05), Authored 2018-2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.2.1 (2020-07-08), Copyright © 2018-2020 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.


Macro Architecture
(System Architecture)
Tier
Area
System
Zone
Program
+
*
+
Meso Architecture
(Software Architecture)
Layer
Slice
Unit
Module
+
*
+
Function
Block
Stroke
Statement
Expression
+
1
*
+
+
+
Micro Architecture
(Programming)


System
Zone
Tier
Area
Area
Tier
Area
Area
Program
Program
Tier
Area
Area
Program
Unit
Slice
Slice
Module
Module
Module
Module
Module
Module
Layer
Module
Module
Function
Block
Stroke
Statement
Statement
Block
Stroke
Statement
Expression

Zone: Structuring **Tier** or **Area** within a **System** (or within an outer **Zone**).

Unit: Structuring **Layer** or **Slice** within a **Program** (or within an outer **Unit**).

Tier: Special kind of ranked **Zone** for horizontally grouping (logically or spatially distinct and network-linked) **Programs** or other **Tiers** and **Areas**.

Layer: Special kind of ranked **Unit** for horizontally grouping (logically distinct and memory-linked) **Modules** or other **Slices** and **Layers**.

Area: Special kind of unranked **Zone** for vertically grouping (logically or spatially distinct and network-linked) **Programs** or other **Tiers** and **Areas**.

Slice: Special kind of unranked **Unit** for vertically grouping (logically distinct and memory-linked) **Modules** or other **Slices** and **Layers**.

System: One or more network-linked **Programs**, forming a solution as a whole and the primary unit of thinking at the level of Macro Architecture.

Module: Know-how encapsulating, potentially reusable group of **Functions** and primary unit of thinking at the level of Meso Architecture.

Program: Stand-alone, self-contained, executable piece of software and primary unit of thinking at the level of both Macro and Meso Architecture.

Function: Operation of a **Module**, with a well-defined interface, and one of the two primary units of thinking at the level of Micro Architecture.

Statement: Execution of a single action, specified through one or more **Expressions** and zero or more inner **Blocks**.

Stroke: Logical sequence of **Statements** in a **Block** and one of the two primary units of thinking at the level of Micro Architecture.

Block: Sequence of one or more **Strokes** within a **Function** or an outer **Statement** and scope for variables of inner **Statements**.

Expression: Simple or complex representation of a single data value.

Eine **Component** ist “any group of anything” in der Software Architecture. Dennoch gibt es prominente Komponentenkategorien, die eine bestimmte allgegenwärtige **Component Hierarchy** im Software Engineering bilden. Diese besteht aus den drei Ebenen **Macro Architecture** (aka System Architecture), **Meso Architecture** (aka Software Architecture) und **Micro Architecture** (aka Programming).

In der Ebene Macro Architecture hat man mit **Systems** (aka Applications) zu tun, die aus hierarchisch angeordneten, infrastrukturellen **Zones** bestehen, welche entweder (horizontale) **Tiers** oder (vertikale) **Areas** sein können. Die **Zones** bestehen ihrerseits aus **Programs**.

Diese **Programs** bestehen in der Ebene Meso Architecture wiederum aus hierarchisch angeordneten **Units**, welche entweder (horizontale) **Layer** oder (vertikale) **Slices** sein können. Die **Units** bestehen ihrerseits aus **Modules**.

Die **Modules** bestehen in der Ebene Meso Architecture wiederum aus **Functions** auf der Ebene Micro Architecture und diese aus hierarchisch angeordneten (lexikalischen) **Blocks**, welche ihrerseits aus **Strokes** (aka “Thoughts”) bestehen, welche ihrerseits aus **Statements** bestehen und diese bestehen am Ende aus einzelnen **Expressions**.

Die fünf **Primary Units of Thinking** sind **Systems**, **Programs**, **Modules**, **Functions** und **Strokes**.

Fragen

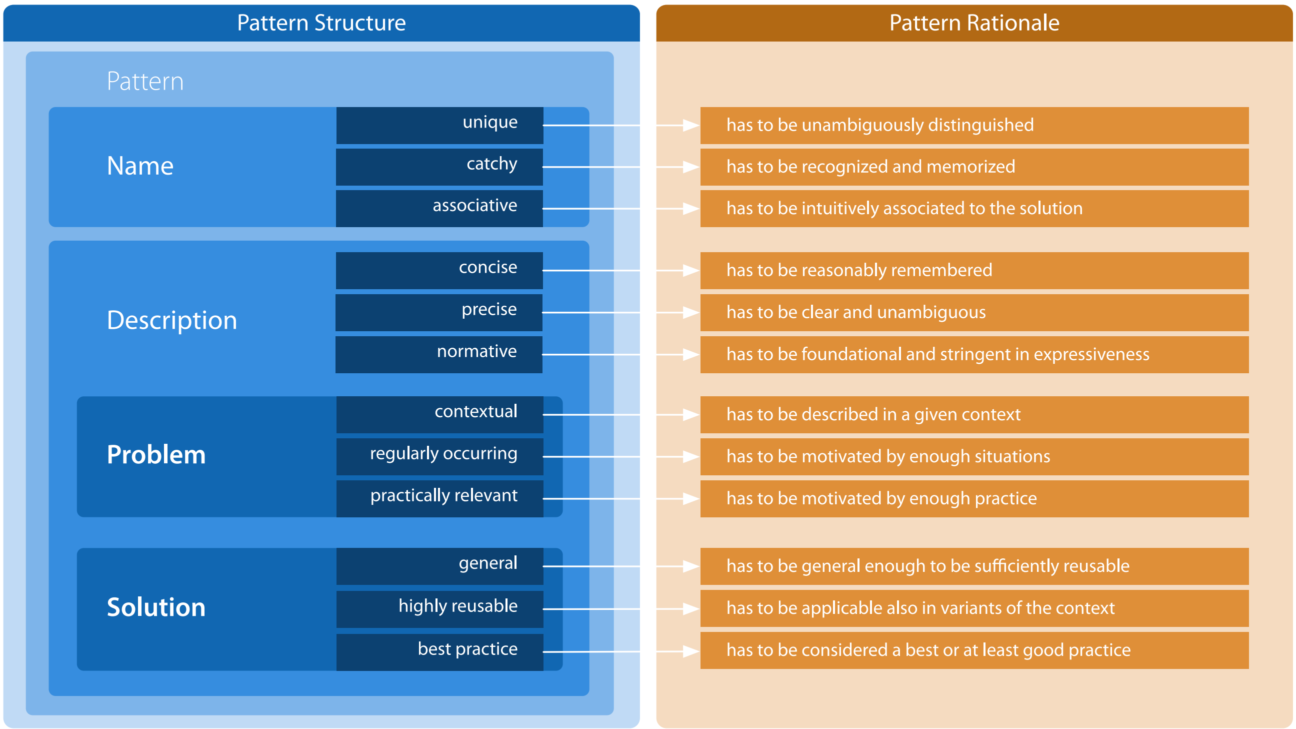
- Welche drei Komponentenkategorien kennt man auf der Ebene der Macro Architecture (aka System Architecture)?
- Welche drei Komponentenkategorien kennt man auf der Ebene der Meso Architecture (aka Software Architecture)?
- Welche fünf Komponentenkategorien kennt man auf der Ebene der Micro Architecture (aka Programming)?

Architecture Patterns

Intellectual Content: Version 1.0.0 (2022-11-05), Authored 2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.0 (2022-11-05), Copyright © 2022 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Pattern Definition

Pattern: unique, catchy, and associative **Name** and concise, precise, and normative **Description** of a contextual, regularly occurring, and practically relevant **Problem** and a general, highly reusable, and best practice **Solution** for it.



Pattern Structure

Pattern

Name
- unique
- catchy
- associative

Description
- concise
- precise
- normative

Problem
- contextual
- regularly occurring
- practically relevant

Solution
- general
- highly reusable
- best practice

Pattern Rationale

- has to be unambiguously distinguished
- has to be recognized and memorized
- has to be intuitively associated to the solution
- has to be reasonably remembered
- has to be clear and unambiguous
- has to be foundational and stringent in expressiveness
- has to be described in a given context
- has to be motivated by enough situations
- has to be motivated by enough practice
- has to be general enough to be sufficiently reusable
- has to be applicable also in variants of the context
- has to be considered a best or at least good practice

Definition eines **Architekturmusters**: einzigartiger, eingängiger und assoziativer Name und prägnante, präzise und normative Beschreibung eines kontextuellen, regelmäßig auftretenden und praktisch relevanten Problems und einer allgemeinen, hochgradig wiederverwendbaren und praxisbewährten Lösung für dieses Problem.

Die Begründungen sind, daß ein **Architekturmuster**: eindeutig unterschieden werden muß, wiedererkannt und erinnert werden muß, intuitiv mit der Lösung assoziiert werden muß, sich gut merken lassen muß, klar und eindeutig sein muß, grundlegend und stringent in der Ausdrucksfähigkeit sein muß, in einem bestimmten Kontext beschrieben werden muß, durch genügend Situationen motiviert sein muß, durch genügend Praxis motiviert sein muß, um ausreichend wiederverwendbar zu sein, auch allgemein genug sein muß, auch in Varianten des Kontexts anwendbar sein muß und als beste oder zumindest gute Praxis angesehen werden muß.

Architekturmuster ermöglichen es einem insbesondere, effizient zu kommunizieren (Name) und von den eingeflossenen Erfahrungen zu profitieren (Best Practice).

Fragen

? Warum sind **Architekturmuster** interessant?

ARCHITECTURE
FUNDAMENTALS

Layer Architectures

(Patterns)

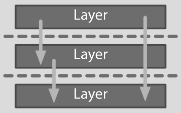
TECHNISCHE
UNIVERSITÄT
MÜNCHEN

Intellectual Content: Version 1.0.5 (2020-11-28), Authored 2018-2020 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.4 (2020-11-28), Copyright © 2018-2020 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Layering Principle

Horizontally split code or data into two or more logically, optionally also spatially, clearly distinct, isolating, named, and ranked Layers.

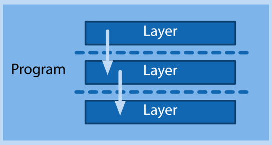
A Layer is not allowed to have relationships to or knowledge about any upper Layers. Additionally, for *Closed Layering*, each Layer is allowed to have relationships to and knowledge about the *directly* lower Layer only. In contrast to *Open Layering* or *Leaky Abstraction*, where each Layer is allowed to have relationships to and knowledge about *any* lower Layer.



LR Layer

Split related code or data of a Program into two or more logically distinct domain- or technology-induced Layers.

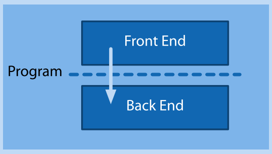
Rationale: Separation of Concern, Single Responsibility Principle, Mastering Complexity, Change Isolation, Functional Abstraction.



FB Front End / Back End

Split the code of a Program into exactly two logical Layers: a user-facing Front End and a data-facing Back End.

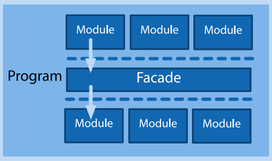
Rationale: Separation of Concern, Single Responsibility Principle, Mastering Complexity, Change Isolation, Functional Abstraction, Organisational Alignment.



FD Facade

Splice a domain-specific Facade Layer into two Layers of two or more Modules. The extra Facade Layer acts as a broker between the Modules.

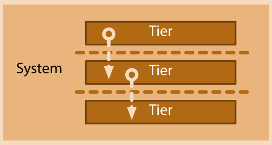
Rationale: Information Hiding, Cross-Cutting Concern Centralization, Functionality Orchestration, Authorization, Validation, Conversion.



TR Tier

Split related code or data of a System into two, three or more logically and spatially distinct, network-connected, domain- or technology-induced Tiers.

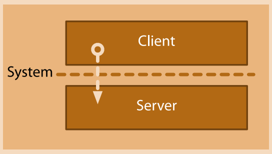
Rationale: Separation of Concern, Single Responsibility Principle, Mastering Complexity, Change Isolation, Functional Abstraction, Deployment Partitioning.



CS Client / Server

Split the code of a System into two spatially distinct, network-connected Layers, each forming a stand-alone Program: a user-facing and multi-instantiated (Rich) Client and a data-facing (and logically) single-instantiated (Thin) Server. Both contain a Front/Back End.

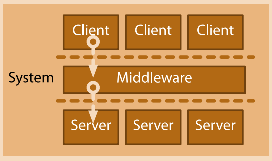
Rationale: Multi-User, User Computing Resource Leverage, Distributed Computing.



MW Middleware

Splice a domain-unspecific Middleware Layer into a Client/Server communication. The extra Layer is a stand-alone Program Tier and acts as a broker between Client and Server.

Rationale: Communication Peer Discovery Simplification, Transport Protocol Conversions, Network Topology Flexibility.



Beim **Layering** werden Code oder Daten in zwei oder mehr logische (**logically**) — ggf. auch "physikalische" (**spatially**) — **Layer** geschnitten. Diese Layer sind klar unterscheidbar (**clearly distinct**), voneinander isoliert (**isolated**), benannt (**named**) und stehen in einer Rangordnung (**ranked**). Layer werden immer **horizontal** gezeichnet.

Ein Layer hat keine Beziehung (**relationship**) zu, oder Wissen (**knowledge**) über, irgendwelche Layer über ihm. Zusätzlich darf er, beim **Closed Layering**, eine Beziehung zu oder Wissen über den direkten Layer unter ihm haben. Außerdem darf er beim **Open Layering** bzw. der **Leaky Abstraction** sogar eine Beziehung zu oder Wissen über irgendwelche Layer unter ihm haben.

Falls sich das Layering über Netzwerk-Grenzen erstreckt oder eine "physikalische" Grenze durchbricht, spricht man nicht mehr von einzelnen Layern, sondern von **Tiers**.

Zerfällt ein Programm in einen vorderen bzw. auf das User-Interface fokussierenden Layer und einen hinteren bzw. auf Daten fokussierenden Layer, so spricht man bei den beiden Layern auch von **Front End** und **Back End** des Programms. Dies ist nicht zu verwechseln mit **Client** und **Server**, welche die zwei Layer (in diesem Kontext tatsächlich Tiers genannt) eines Systems über ihre speziellen Rollen benennen. Sowohl der Client als auch der Server sind eigenständige Programme mit jeweils einem Front End und einem Back End.

Ein sehr spezieller und prominenter Layer ist die **Facade**, welche innerhalb eines Programms die Module von zwei anderen Layern voneinander trennt. Eine Variante der Facade auf Ebene eines Systems (statt auf der Ebene eines Programms) ist die **Middleware**, welche eine Client/Server-Kommunikation auftrennt.

Fragen

- ? Wie nennt man die entstehenden Einheiten, wenn Code oder Daten **horizontal** geschnitten werden?
- ? Was ist der Unterschied zwischen den Layer-Pärchen **Front/Back End** und **Client/Server**?

Slice Architectures

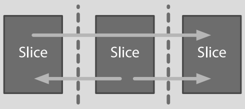
(Patterns)

Intellectual Content: Version 1.0.5 (2024-04-15), Authored 2018-2024 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.2 (2019-09-30), Copyright © 2018-2019 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Slicing Principle

Vertically split code or data into two or more logically, optionally also spatially, clearly distinct, named, and unranked slices.

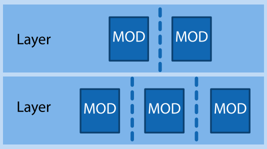
The particular slicing should minimize the total amount of individual relationships between the resulting slices. Per type of relationship, there should be no cycle in the transitive relationships.



MOD Concerned Module

Split related code or data (usually across a single Layer) into two or more logically distinct domain- or technology-induced Modules.

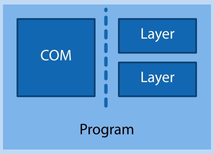
Rationale: Separation of Concern, Single Responsibility Principle, Mastering Complexity.



COM Common Slice

Factor out common or cross concern code or data of a Program (across all Layers) into a single spatially distinct, separate slice.

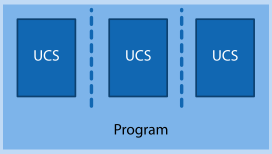
Rationale: Lack of Redundancy, Single Point of Truth, Reusability.



UCS Use-Case Slice

Split the code and data of a Program (across all Layers) into two or more purely logical slices, one for each distinct, domain-specific Use-Case.

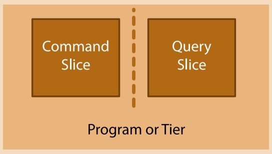
Rationale: Comprehensibility, Domain Alignment, Mastering Complexity.



CQRS Command-Query Responsibility Segregation

Split code and data of a Program (across all Layers) or a Tier into exactly two slices to segregate operations that read data (queries) from the operations that update data (commands).

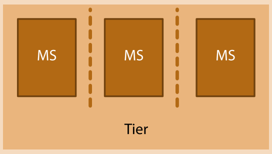
Rationale: Separated Scalability, Separated Data Access Patterns, Event Sourcing Approach.



MS Microservice

Split code and data of a Tier (across all Layers) into two or more distinct, loosely-coupled, domain-enclosed, functional services, each forming a stand-alone Program.

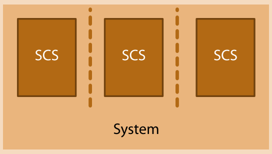
Rationale: Heterogeneity, Long-Term Large-Scale Maintenance, Replaceability, Resilience, (Scalability), (Easy Deployment), (Organizational Alignment), (Composability), (Reusability).



SCS Self-Contained System

Split code and data of a System (across all Layers and Tiers) into two or more distinct, loosely-coupled, domain-enclosed, functional systems, each forming a stand-alone sub-System.

Rationale: Mastering Complexity, Heterogeneity, Resilience, Scalability, Easy Deployment, Organizational Alignment, Reusability, Replaceability.



Beim **Slicing** werden Code oder Daten in zwei oder mehr logische (**logically**) — ggf. auch "physikalische" (**spatially**) — **Slices** geschnitten. Diese Slices sind klar unterscheidbar (**clearly distinct**), voneinander isoliert (**isolated**) und benannt (**named**). Slices werden immer **vertikal** gezeichnet.

Slices im selben Layer sollten möglichst unabhängig voneinander sein. Im Falle von Beziehungen sollte zumindest kein Zyklus existieren. Es gibt verschiedene spezielle Varianten von Slices, die jeweils einen eigenen Namen haben.

Concerned Modules sind Slices eines Layers, welche ein bestimmtes fachliches oder technisches Anliegen umsetzen. **Common Slice** ist ein Slice eines Programs, wohin Gemeinsamkeiten anderer Layer verschoben wurden. **Use-Case Slices** sind Slices eines Programs, die sich jeweils dediziert um bestimmte fachliche Anwendungsfälle kümmern.

Bei der **Command-Query Responsibility Segregation** Architektur wird ein Tier in zwei Slices für Commands/ Writes und Queries/Reads getrennt. Ein **Microservice** ist ein Slice eines Tiers, welcher als getrenntes Programm ausgeführt ist und sich um eine abgeschlossene fachliche Funktionalität kümmert. Ein **Self-Contained System** ist ein Slice eines gesamten Systems, welches als getrenntes Sub-System ausgeführt ist.

Fragen

- ? Wie nennt man die entstehenden Einheiten, wenn Code oder Daten **vertikal** geschnitten werden?
- ? Wie nennt man die Slices eines Tiers, welche als getrennte Programme ausgeführt sind, die sich jeweils um bestimmte fachlich abgeschlossene Funktionalitäten kümmern?

ARCHITECTURE
FUNDAMENTALS

Flow Architectures
(Patterns)

TECHNISCHE
UNIVERSITÄT
MÜNCHEN

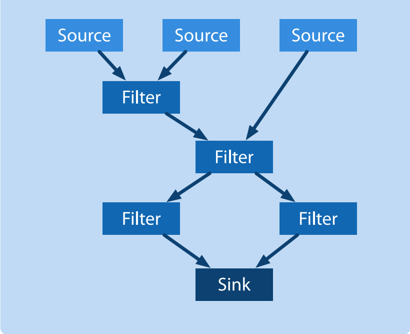
AF 07.1

Intellectual Content: Version 1.0.2 (2018-10-29), Authored 2018 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.3 (2018-11-24), Copyright © 2018 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Pipes & Filters

Pass data through a directed graph of **Components** and connecting **Pipes**. The components can be **Sources**, where data is produced, **Filters**, where data is processed, or **Sinks**, where data is captured. Source and Filter components can have one or more output Pipes. Filter and Sink components can have one or more input Pipes. Components are independent processing units and operate fully asynchronously.

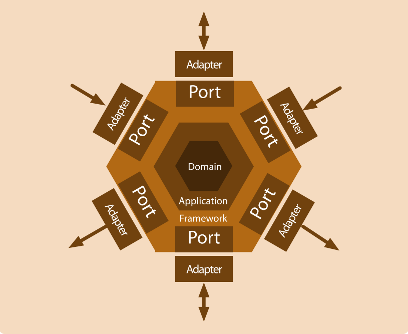
Examples: Unix commands with stdin/stdout/stderr and the Unix shell connecting them with pipes; Apache Spark or Apache Camel data stream processing pipelines.



Ports & Adapters (Hexagonal)

Perform communication in a Hub & Spoke fashion by structuring a solution into the three "Layers" **Domain**, **Application** and **Framework** and use the Framework layer to connect with the outside through **Ports** (general Interfaces) and **Adapters** (particular Implementations). Often some Ports & Adapters are user-facing sources and some are data-facing sinks, although the motivation for the Ports & Adapters architecture is to remove this distinction between user and data sides of a solution.

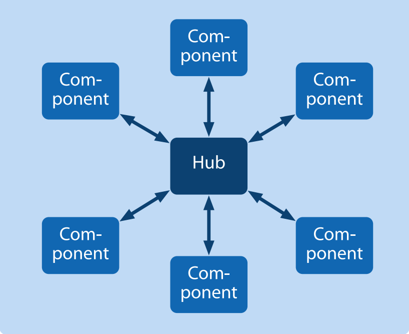
Examples: Message Queue, Enterprise Service Bus or Media Streaming Service internal realization.



Hub & Spoke

Perform communication (the **Spoke**) between multiple Components through a central **Hub** Component. Instead of having to communicate with N x (N-1) / 2 bi-directional interconnects between N Components, use the intermediate Hub to communicate with just N interconnects only. Sometimes one distinguishes between K (0 < K < N) source and N - K target Components and then K x (N - K) uni-directional interconnects are reduced to just N interconnects, too.

Examples: Message Queue, Enterprise Service Bus, Module Group Facade, GNU Compiler Collection, ImageMagick, etc.



Bei den **Flow Architectures** geht es um den primären Datenfluss bzw. die primäre Kommunikation einer Anwendung. Hier gibt es u.a. die drei folgenden klassischen Architekturansätze.

Bei **Pipes & Filters** wird ein gerichteter **Graph** aufgebaut. Die Knoten des Graphen sind die **Components**, welche entweder vom Typ **Source** (Quelle), **Filter** (Filter) oder **Sink** (Senke) sind. Die Kanten des Graphen sind die **Pipes**: die Datenübertragungsstrecken zwischen den **Components**.

Bei dem speziellen **Ports & Adapters** (aka **Hexagonal Architecture**) wird eine "Hub & Spoke" Struktur aufgebaut. Der "Hub" sind Components des Anwendungskerns. Die "Spokes" bestehen jeweils aus einer **Component**, welche sich aus dem **Port** (das Interface) und dem **Adapter** (die Implementation) zusammensetzt.

Bei **Hub & Spoke** allgemein geht es darum, daß eine zentrale **Hub**-Component als Kommunikationsmittelpunkt zwischen sternförmig um den Hub angeordneten Components agiert. Der Knackpunkt dabei ist, daß die N x (N - 1) / 2 Kommunikationswege zwischen Components dank der **Hub**-Component auf nur N Kommunikationswege (**Spokes**) reduziert werden.

Fragen

- ? Mit Hilfe welcher **Flow Architecture** kann man N Komponenten so miteinander verschalten, daß statt N x (N - 1) / 2 nur N Kommunikationswege entstehen?

ARCHITECTURE FUNDAMENTALS

Process Architectures

(Patterns)

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 07.2

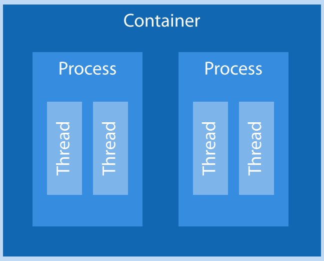
Intellectual Content: Version 1.0.2 (2019-09-21), Authored 2018-2019 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.1 (2018-11-24), Copyright © 2018 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Container, Process, Thread

The Operating System manages and orchestrates the run-time execution of applications in **Containers**, programs in **Processes** and control flows in **Threads**.

Containers are the ultimate enclosures, separating and controlling both the computing resources processor, memory, storage and network. Processes are the primary enclosures, still separating and controlling at least the computing resources processor and memory. Threads are the light-weight enclosures, just separating and controlling the computing resource processor. Containers can contain one or more Processes, and Processes can contain one or more Threads.

Examples: Docker Container,
Unix Processes, POSIX Threads.

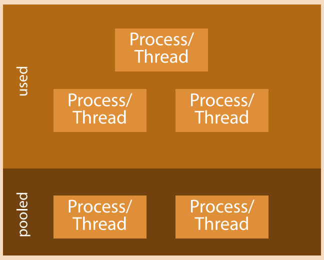


Process/Thread Pool

Instead of creating a Process/Thread for handling each incoming I/O request, pick a pre-created Process/Thread out of a resource **Pool** in order to increase performance and decouple I/O traffic (leading to threads of execution) from the actual computing resource usage and utilization.

The Process/Thread Pool usually has a lower and upper bound of processes/threads. The lower bound keeps the system "hot" between I/O requests. The upper bound limits the computing resource usage and avoids over-utilization.

Examples: Apache HTTP Daemon

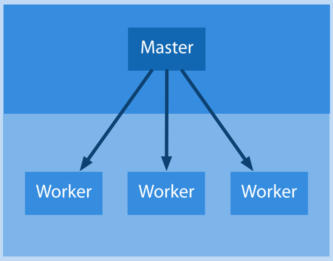


Master-Worker

The system has a single permanent **Master** container/process/thread and a Pool of many ephemeral **Worker** containers/processes/threads. The Master starts, restarts, pauses, resumes and stops the Workers and usually also delegates incoming I/O requests to them. The Workers process the I/O requests and deliver the responses.

Starting the Master usually implicitly starts an initial set of Workers (the initial Pool), stopping the Master implicitly stops all still pending Workers.

Examples: Unix init(8) daemon, Apache HTTP Daemon, SupervisorD, Node.js Cluster module



Bei den **Process Architectures** geht es um das Zusammenspiel verschiedener **Container**, **Processes** oder **Threads**. Alle drei Konzepte kapseln Code und Daten. **Container** sind die stärkste Kapselung, bei der die Nutzung von sowohl CPU, RAM, Festplattenspeicher und Netzwerk gekapselt wird (z.B. Docker Container). Bei einem **Process** wird die Nutzung von CPU und RAM gekapselt (z.B. Unix Prozess). Bei einem **Thread**, der schwächsten Kapselung, wird nur noch die Nutzung der CPU gekapselt (z.B. Unix Thread).

Um gleichzeitig mehrere Anfragen beantworten zu können, werden in Server-Anwendungen mehrere Processes/Threads pro Anfrage genutzt. Da das ständige Erzeugen von solchen Processes/Threads nennenswert die Runtime-Performance reduziert und man die Hardware-Last üblicherweise limitieren und nicht vollkommen linear an die eingehenden Anfragen koppeln möchte, wird üblicherweise ein sogenannter **Pool** von einmalig erzeugten Worker-Processes/Threads eingesetzt (z.B. Apache HTTPd oder NGINX).

Klassisch wird ein solcher Pool in einen **Master** Process/Thread und viele **Worker** Processes/Threads unterteilt. Der permanent laufende Master erzeugt, steuert und stoppt die flüchtigen Worker. Im Falle von Fehlern werden die Worker vom Master ggf. aktiv gestoppt oder bei einem Crash automatisch neu gestartet (z.B. Node.js `cluster` Modul).

Fragen

- ? Mit welcher **Process Architecture** wird in der Praxis üblicherweise ein Process/Thread **Pool** verwaltet?

ARCHITECTURE FUNDAMENTALS

Cluster Architectures

(Patterns)

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 07.3

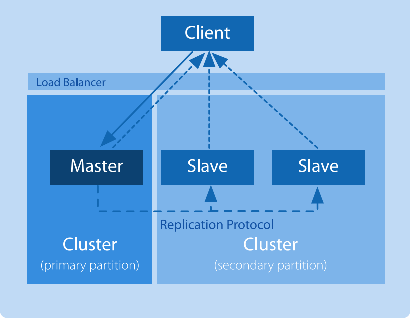
Intellectual Content: Version 1.0.5 (2021-09-08), Authored 2018-2021 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.4 (2021-09-08), Copyright © 2018-2021 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Master-Slave (Static Replication)

Cluster of a single **Master** and multiple **Slave** nodes, where data is continuously copied from the Master to the Slave nodes in order to support high-availability (where a Slave will take over the Master role) in case of a Master outage and increased read performance (where regular read requests are also served by the Slaves).

In this static replication scenario the Master is usually assigned statically and in case of outages has to be reassigned usually semi-manually. Especially, the full reestablishment of the original Master assignment after a Master recovery usually is a manual process.

Examples: OpenLDAP Replication, PostgreSQL WAL Replication.

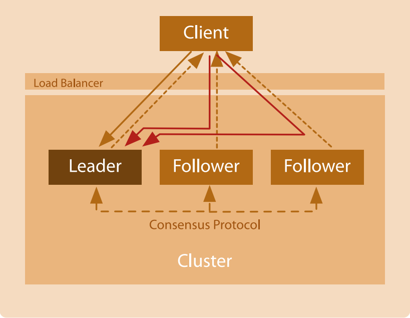


Leader-Follower (Dynamic Replication)

Cluster of a single **Leader** and multiple **Follower** nodes, where data is written on the current Leader node and data read on both the current Leader and all Follower nodes. For writing data to the cluster, the Leader node performs a consensus protocol (e.g. RAFT, Paxos or at least Two-Phase-Commit) with the Followers and this way automatically and consistently replicates the data to the Followers.

In this dynamic replication scenario the Leader is usually automatically assigned by the cluster nodes through an election protocol and in case of outages is automatically re-assigned. There is usually no re-establishment of the original Leader assignment.

Examples: Apache Zookeeper, Consul, EtcD, CockroachDB, InfluxDB.

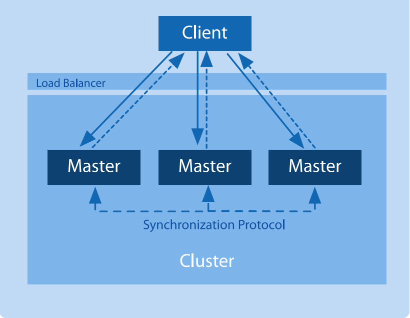


Master-Master (Synchronization)

Cluster of multiple **Master** nodes, where data is read and written on any Master node concurrently. The Master nodes either use Strict Consistency through writing to a mutual-exclusion-locked shared storage concurrently or use Eventual Consistency in a Shared Nothing storage scenario where they continuously synchronize their local data state to all other nodes with the help of a synchronization protocol.

The synchronization protocol usually is based on either Conflict-Free Replicated Data Types (CRDT) or at least Operational Transformation (OT). In any scenario, data update conflicts are explicitly avoided.

Examples: ORACLE RAC, MySQL/MariaDB Galera Cluster, Riak, Automerge/Hypermerge.



→ Write Operation
⇢ Read Operation

Bei den **Cluster Architectures** wird der Zusammenschluss von Rechnerknoten zu einem Cluster adressiert.

Bei der **Master-Slave** Architecture handelt es sich um eine statische Replikation der Daten von einem Master Server auf einen oder mehrere Slave Server. Die Clients können Lesezugriffe an alle Server senden, müssen aber Schreibzugriffe ausschließlich über den Master laufen lassen. Dies wird üblicherweise dazu verwendet, um die Read Performance zu steigern.

Bei der **Leader-Follower** Architecture handelt es sich um eine Art dynamische Replikation der Daten von einem Leader Server auf einen oder mehrere Follower Server. Die Clients können sowohl Lese- als auch Schreibzugriffe an alle Server senden. Da nur der Leader Server die Schreibzugriffe bearbeiten darf, leiten die Follower Server diese, intern und für den Client intransparent, an den Leader Server weiter.

Dies ist auch der Unterschied zu Master-Slave: der Leader wird über ein Leader Election Protocol (bei einem Ausfall des aktuellen Leader Servers) automatisch und dynamisch zwischen allen Servern festgelegt. Der Vorteil ist, daß sich Leader-Follower für die Clients wie Master-Master anfühlt, der Cluster aber keine aufwändige Konfliktlösungsstrategie wie bei Master-Master umsetzen muss.

Bei der **Master-Master** Architecture handelt es sich um eine echte Synchronisation der Daten zwischen zwei oder mehr gleichberechtigten Master Servern. Die Clients können sowohl Lese- als auch Schreibzugriffe an beliebige Master Server richten. Die Master Server müssen Konflikte vermeiden (üblicherweise durch verteilte Transaktionen), welche durch gleichzeitige Änderungen an denselben Daten auftreten können.

Fragen

? Welche einfache **Cluster Architecture** kann man einsetzen, wenn die Read-Performance einer Server-Anwendung gesteigert werden soll?

ARCHITECTURE FUNDAMENTALS

Networking Architectures

(Patterns)

TECHNISCHE UNIVERSITÄT MÜNCHEN

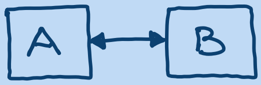
AF 08.1

Intellectual Content: Version 1.0.7 (2022-10-09), Authored 2018-2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.4 (2019-10-31) Copyright © 2018-2019 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

PTP Point-to-Point

Communicate between two network nodes in a point-to-point fashion, usually through a direct link.

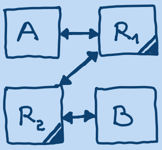
Rationale: simple communication where both nodes know about each other and can directly reach each other.



RTG Routing

Communicate between two network nodes in a point-to-point fashion, but by routing the network packets over intermediate forwarding nodes (routers).

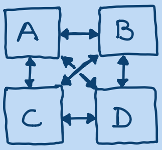
Rationale: simple communication where both nodes know about each other, but cannot directly reach each other.



P2P Peer-to-Peer

Communicate between multiple network nodes (usually all in the client and server role at the same time) without involving a central hub node (in the role of a server) — except for the initial network entry discovery.

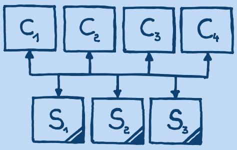
Rationale: communication without central control (although a seed peer is required).



C/S Client/Server

Communicate between multiple nodes in the client role (making requests, and usually with ephemeral addresses) and multiple nodes in the server role (serving responses, and usually with fixed addresses).

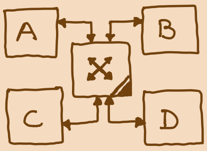
Rationale: communication with central orchestration, control and data storage.



BUS Bus/Broker/Relay

Communicate between multiple nodes with the help of a central packet forwarding hub node in a star network topology.

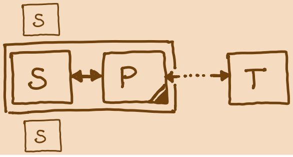
Rationale: decouple communication nodes: instead of Point-to-Point (PTP) communications between all nodes, there are just PTP communications with the hub.



FPR (Forward) Proxy

Communicate between two nodes by using an intermediate forwarding proxy node in front of the source node.

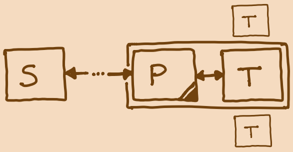
Rationale: bridge network topology constraints (segmented networks); caching at source side; auditing of communication.



RPR Reverse Proxy

Communicate between a source and a target node by using a masquerading proxy node directly in front of the target node.

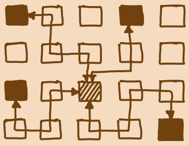
Rationale: load balancing for multiple target nodes; caching at target side; auditing of communication; security shielding of target nodes; protocol conversions.



VPN Virtual (Private) Network

Communicate between nodes in a logical star network topology on top of an arbitrary physical routed network topology.

Rationale: secure private network overlaying an unsecure public network; simplify network topology.



Bei den **Networking Architectures** wird die netzwerktopologische Kommunikation zwischen Rechnerknoten adressiert. Der einfachste Weg ist eine **Point-to-Point** Kommunikation über eine direkte Verbindung der Knoten.

Üblicherweise geht die Kommunikation heute aber über ein Netzwerk von Knoten, bei dem die einzelnen Nachrichten mit Hilfe von **Routing** über Zwischenknoten ausgetauscht werden.

Kommunizieren alle Knoten in sowohl Client- als auch Server-Rolle direkt miteinander, so spricht man von einer **Peer-to-Peer** Architektur. Sind einige Knoten nur in der Client-Rolle und andere nur in der Server-Rolle, so spricht man von einer **Client/Server** Architektur.

Um mehrere Knoten miteinander kommunizieren zu lassen, ohne daß diese sich jeweils kennen und adressieren müssen, nutzt man üblicherweise einen zentralen **Bus/Broker** Knoten und eine Stern-Topologie.

Sind zwischen Quelle (Source) und Ziel (Target) Zwischenknoten aktiv, welche als Stellvertreter (**Proxy**) in der Kommunikation agieren und nicht nur wie ein **Router** die Netzwerk-Pakete weiterleiten, spricht man entweder von einer **(Forward) Proxy** oder **Reverse Proxy** Situation. Ersteres, wenn der Proxy auf Seite des Quellknotens agiert, letzteres, wenn der Proxy als Stellvertreter des Zielknotens agiert.

Zusätzlich kann ein sogenanntes **Virtual Private Network** etabliert werden, bei dem ein logisches abgesichertes “Overlay Network” über ein physikalisches Netzwerk gelegt wird.

Fragen

- ? Mit welcher **Network Architecture** kann man mehrere Knoten miteinander kommunizieren lassen, ohne daß diese Knoten sich jeweils untereinander genau kennen müssen?
- ? Wie nennt man einen Rechnerknoten, der als Stellvertreter für einen Zielknoten agiert?